

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра прикладной математики

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Технологии и инструментарий анализа больших данных»

Практическая работа №3
Распределенная модель обработки данных

Студент группы ИНБО-12-23, Албахтин И.В. _____
(подпись)

Преподаватель Приходько Н.А. _____
(подпись)

Москва 2026 г.

Содержание

1 Цель и постановка задачи 3
2 Окружение и источники 3
3 Схемы и преобразования 4
4 Пропуски и предметная интерпретация 5
5 Агрегации и соединения 6
6 Spark SQL и сохранение витрины 7
7 Самостоятельный анализ продаж КОМУС 9
7.1 Чтение, схема и преобразования 9
7.2 Качество данных и JOIN 10
7.3 Агрегации, Spark SQL и экспорт 11
8 Воспроизводимость и выводы 14
Ответы на контрольные вопросы 15
Источники 18

1 Цель и постановка задачи

Цель работы — выполнить распределённую табличную обработку в PySpark: подготовить источник SQLite, задать схемы, исследовать пропуски, построить агрегаты и JOIN, сопоставить Spark SQL с DataFrame API и проверить экспорт. Учебная часть выполнена на Olist, самостоятельная — на продажах КОМУС.

Итогом стали аналитическая витрина Olist из 74 категорий и витрина КОМУС по товарным группам. Результаты получены полным последовательным запуском notebook в новом ядре; исходные таблицы не изменялись.

2 Окружение и источники

Результаты приведены в таблице 1.

Таблица 1 – Окружение выполнения

Параметр	Значение
VM / ОС	TIABD-Albakhtin-IV / Ubuntu Server 22.04.5 LTS
Ресурсы	2 vCPU; 4 ГБ RAM; диск 40 ГБ
Python	3.10.12
Java	openjdk version "17.0.20.1" 2026-08-18
Spark	3.5.7
Master / parallelism	local[*] / 2
Driver memory	1g
Shuffle partitions / AQE	8 / true

Local[*] использует два виртуальных ядра одного узла. Методичка рекомендует Python 3.11; фактически применены совместимые Python 3.10, Spark 3.5.7 и JDK 17. Локальный режим сохраняет модель partitions, transformations и actions, но не проверяет сеть физического кластера.

Ecommerce.sqlite восстановлена из CSV Olist, использованных во второй практике, поскольку исходный файл SQLite не был передан. Восстановление и staging выполняются отдельным скриптом. Фактическая версия содержит 100 000 отзывов, тогда как в примере методички указано 97 621; эта разница не скрывается и не исправляется под ожидаемые числа. Справочник категорий восстановлен до исходных 71 записей.

Результаты приведены в таблице 2.

Таблица 2 – Выгруженные учебные таблицы

Таблица	Число строк
orders	99 441
order_items	112 650
products	32 951
customers	99 441
order_payments	103 886
order_reviews	100 000
product_category_name_translation	71

Spark не читает SQLite как штатный файловый формат. Модуль sqlite3 и Pandas выгружают таблицы частями по 50 000 строк; последующая обработка производится в Spark. Данные КОМУС получены из предоставленного архива «Транзакции КОМУС.zip». Клиентские реквизиты, адреса и номера машин исключены ещё при подготовке входа. Архивы transactions и northwind_small не использовались: выбранный набор КОМУС удовлетворяет требованиям самостоятельной части.

Вход учебной части: ecommerce.sqlite → staging CSV → связанные DataFrame orders, order_items, products, customers, payments и translations. Самостоятельная часть: продажи КОМУС → fact; товары КОМУС → dim; соединение fact.product_key = dim.product_key. Связи orders–customers и items–products контролируются по идентификаторам, а не по названиям.

3 Схемы и преобразования

Orders при чтении CSV имеет строковые столбцы. Для order_items задан StructType: order_item_id — integer, price и freight_value — double, идентификаторы — string. Автоматический вывод схемы дополнительно определяет shipping_limit_date как timestamp, поэтому схемы не обязаны полностью совпадать. Пять временных полей orders преобразованы функцией to_timestamp.

```
items_schema = StructType([
   StructField("order_id", StringType(), True),
   StructField("order_item_id", IntegerType(), True),
   StructField("product_id", StringType(), True),
   StructField("seller_id", StringType(), True),
   StructField("shipping_limit_date", StringType(), True),
   StructField("price", DoubleType(), True),
   StructField("freight_value", DoubleType(), True)
])
```

Применены select, filter, withColumn, drop, dropDuplicates и orderBy. Для позиций price ≥ 500 вычислена цена с доставкой gross_item_value; условию удовлетворяют 3 239 строк. Это стоимость позиции, а не всего заказа.

Результаты приведены в таблице 3.

Таблица 3 – Самые дорогие позиции с доставкой

Позиция	Цена	Доставка	Итого
1	6 735,00	194,31	6 929,31
1	6 729,00	193,21	6 922,21
1	6 499,00	227,66	6 726,66
1	4 799,00	151,34	4 950,34
1	4 690,00	74,34	4 764,34

Вывод: строковые временные значения приведены к timestamp; отбор дорогих позиций сохраняет уровень наблюдения order_id + order_item_id. Результаты printSchema() и сравнение с inferSchema сохранены в выполненном notebook.

4 Пропуски и предметная интерпретация

Результаты приведены в таблице 4.

Таблица 4 – Пропуски временных полей orders

Поле	Null
order_purchase_timestamp	0
order_approved_at	160
order_delivered_carrier_date	1783
order_delivered_customer_date	2965
order_estimated_delivery_date	0

Фактическая дата доставки отсутствует в 2 965 заказах. По статусам это 1 107 shipped, 619 canceled, 609 unavailable, 314 invoiced, 301 processing, 8 delivered, 5 created и 2 approved. Дата не заполнена вымышленным значением и не служит причиной удаления незавершённых заказов.

Из 96 478 заказов delivered дата известна у 96 470. Просрочено 7 826 заказов, или 8,11% известной выборки. В products у 610 товаров отсутствует категория; она обозначена unknown, чтобы сохранить позиции при соединении.

На рисунке 1 показан результат выполнения на стенде.

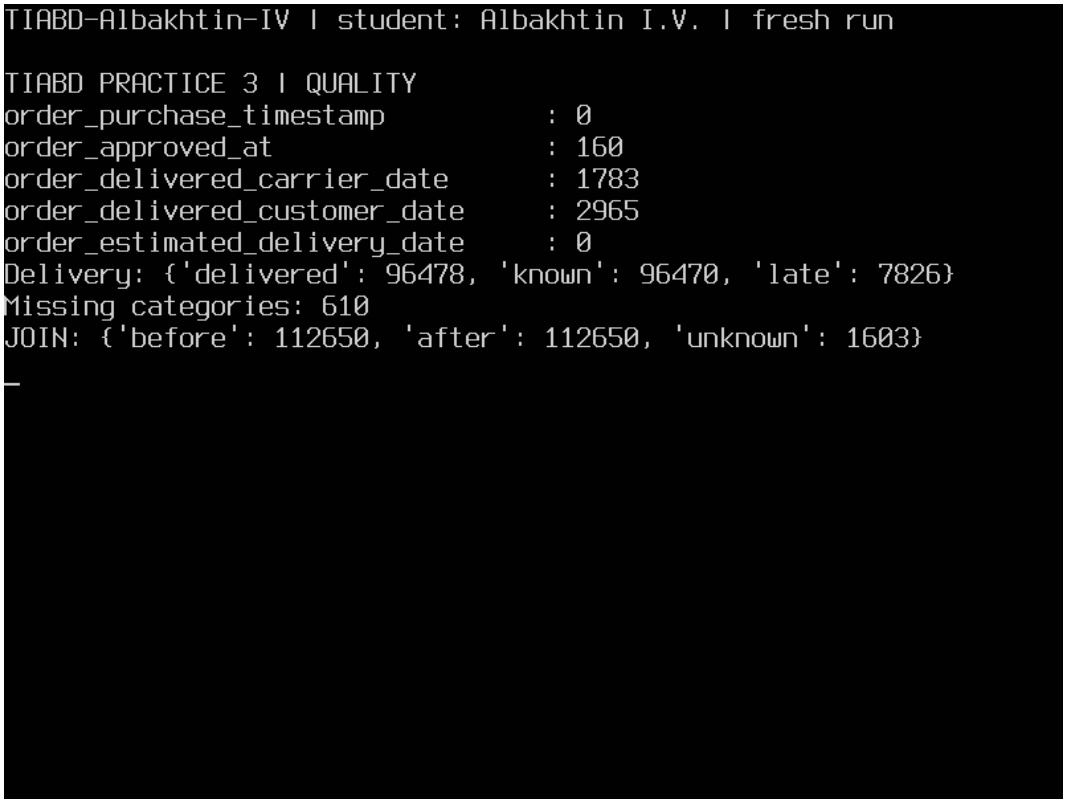


Рисунок 1 – Проверка пропусков и сохранения строк в консоли Ubuntu

5 Агрегации и соединения

Результаты приведены в таблице 5.

Таблица 5 – Статусы заказов

Статус	Заказы
delivered	96 478
shipped	1 107
canceled	625
unavailable	609
invoiced	314
processing	301
created	5
approved	2

Результаты приведены в таблице 6.

Таблица 6 – Агрегаты платёжных строк

Способ оплаты	Строки	Сумма	Среднее
credit_card	76 795	12 542 084,19	163,32
boleto	19 784	2 869 361,27	145,03
voucher	5 775	379 436,87	65,70
debit_card	1 529	217 989,79	142,57
not_defined	3	0,00	0,00

Платежи credit_card преобладают по числу строк и сумме. COUNT(*) в order_payments нельзя интерпретировать как число заказов: один заказ может иметь несколько платежей.

LEFT JOIN order_items → products → translations сохранил 112 650 из 112 650 позиций; 1 603 позиций имеют unknown. Категория выбирается через coalesce: английский перевод, исходное название, затем unknown.

Результаты приведены в таблице 7.

Таблица 7 – Пять штатов с высокой долей просрочки

Штат	Доставлено	Просрочено	Доля %
AL	397	95	23,93
MA	717	141	19,67
PI	476	76	15,97
CE	1279	196	15,32
SE	335	51	15,22

AL имеет 23,93% просрочки при 397 известных доставках. Доля сравнивается вместе с размером группы; сама таблица не доказывает причинное влияние штата на доставку.

6 Spark SQL и сохранение витрины

```
sales.createOrReplaceTempView("sales")
SELECT category, COUNT(*) AS items,
    COUNT(DISTINCT order_id) AS orders,
    ROUND(SUM(price), 2) AS revenue,
    ROUND(AVG(price), 2) AS avg_price
FROM sales
GROUP BY category
ORDER BY revenue DESC
```

Результаты приведены в таблице 8.

Таблица 8 – Топ категорий Olist по сумме price

Категория	Позиции	Сумма price	Средняя цена
health_beauty	9 670	1 258 681,34	130,16
watches_gifts	5 991	1 205 005,68	201,14
bed_bath_table	11 115	1 036 988,68	93,30
sports_leisure	8 641	988 048,97	114,34
computers_accessories	7 827	911 954,32	116,51

Двусторонний exceptAll между полными результатами SQL и DataFrame API дал 0 различий. Витрина содержит 74 группы и записана в Parquet и CSV; для маленького CSV использован coalesce(1). После чтения Parquet совпали число строк, имена и типы полей. Признак nullable стал true для ранее гарантированно заполненных агрегатов, поэтому полное равенство StructType отдельно отмечено как false.

Результаты приведены в таблице 9.

Таблица 9 – Схема итоговой витрины Olist

Поле	Тип до / после Parquet	Nullable до / после
category	string / string	False / True
items	long / long	False / True
orders	long / long	False / True
revenue	double / double	True / True
avg_price	double / double	True / True

На рисунке 2 показан результат выполнения на стенде.

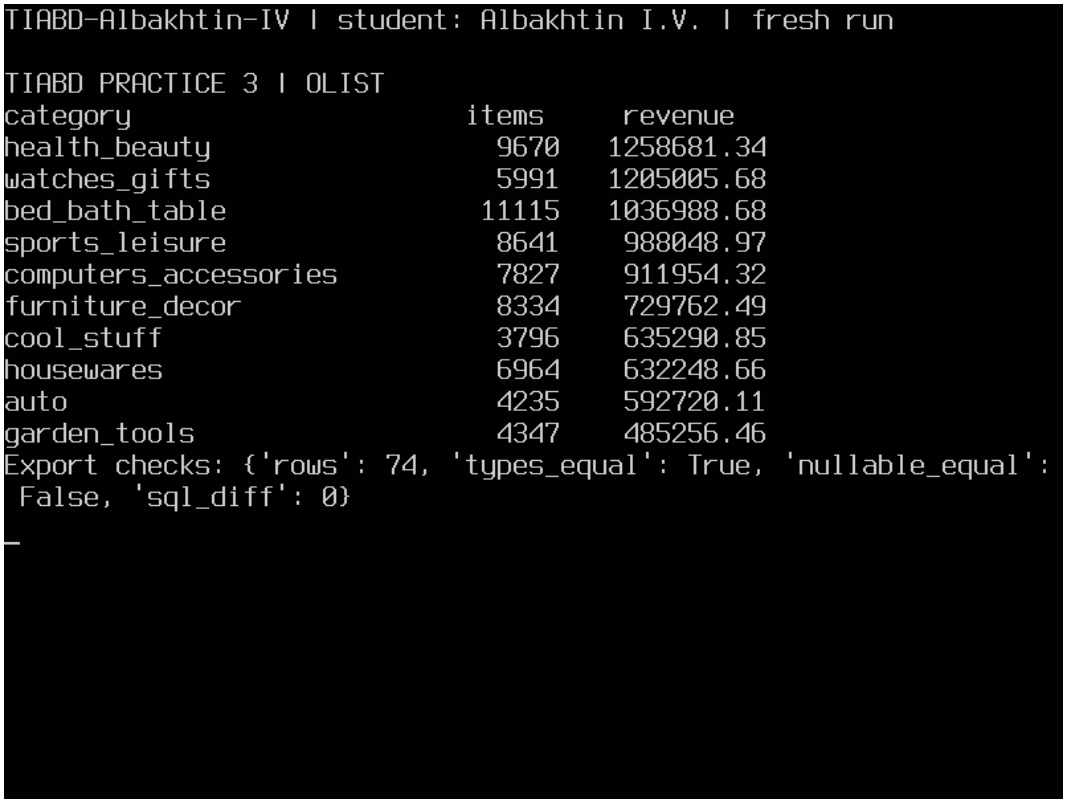


Рисунок 2 – Вычисленные категории Olist и проверки экспорта

7 Самостоятельный анализ продаж КОМУС

Основная таблица содержит 865 222 строк и восемь полей: себестоимость, дата, чек, товарный ключ, филиал, количество, сумма покупки и скидка. Справочник содержит 21 163 уникальных ключей товара и признаки бренда и группы. Названия и условия предоставлены в пользовательском архиве; публичная лицензия на исходные строки не заявлена, поэтому в Git сохраняются код, агрегаты и обезличенные результаты, а не исходные транзакции.

7.1 Чтение, схема и преобразования

```
raw = (spark.read.option("header", True)
   .option("escape", "").option("mode", "FAILFAST")
   .schema(own_schema).csv(str(own_path / "sales.csv")))
dim_raw = (spark.read.option("header", True)
   .option("escape", "").option("mode", "FAILFAST")
   .csv(str(own_path / "products.csv")))
```

CSV записан Pandas с двойной кавычкой как экранирующим символом; этот же символ явно задан читателю Spark. Все четыре поля 21 163 товаров

сверены со стандартным csv.DictReader: расхождений 0. В полном справочнике 464 разных групп; продажи относятся к 437 из них. Проверка числа строк JOIN дополняется проверкой содержимого справочника.

Результаты приведены в таблице 10.

Таблица 10 – Типы полей продаж КОМУС

Поле	Сырой вход	После преобразования
cost	string	double
purchase_date	string	date
receipt_id	string	string
product_key	string	string
branch	string	string
quantity	string	double
amount	string	double
discount	string	double

Применены select, filter, withColumn, dropDuplicates и orderBy. Содержательные признаки: month для динамики, margin = amount − cost для строк с известной себестоимостью. Категориальные признаки филиала, бренда и группы используются отдельно от денежных величин.

Явный StructType фиксирует строковую схему сырого входа. Дата преобразована из dd.MM.yyyy, десятичная запятая заменена точкой перед приведением четырёх числовых полей. Филиал очищен trim, вычислены месяц покупки и margin = amount − cost. Категориальные пропуски справочника маркируются unknown только в соответствующих полях.

7.2 Качество данных и JOIN

Результаты приведены в таблице 11.

Таблица 11 – Пропуски КОМУС после приведения типов

Поле	Null
receipt_id	3 934
purchase_date	0
amount	0
cost	86 431
product_key	0

У 3 934 строк отсутствует номер чека, но продажи сохранены: фиктивный чек не создаётся. У 86 431 строки отсутствует себестоимость; такие строки участвуют в выручке, но их маржа остаётся null. SUM(margin)

описывает только строки с известной себестоимостью, а cost_known показывает охват. Фильтр валидной даты и суммы не удалил строки, поскольку оба поля полностью заполнены.

До JOIN: 865 222 строк; LEFT JOIN: 865 222; INNER JOIN: 865 222. Несопоставленных ключей — 0, конфликтующих ключей справочника — 0. Полный охват объясняет одинаковые размеры обоих соединений.

7.3 Агрегации, Spark SQL и экспорт

Результаты приведены в таблице 12.

Таблица 12 – Товарные группы КОМУС с высокой выручкой

Группа	Число строк	С cost	Выручка	Известная маржа	Средняя сумма
Основы-холст на подрамнике-Базовый	6 703	6 010	4 407 000,88	2 022 021,65	657,47
z-Уценка	37 800	34 117	3 296 317,47	1 478 030,62	87,20
Живопись-акварель-наборы	6 582	5 930	3 182 778,15	1 447 107,19	483,56
Бумага-Картон-пенокартон	2 531	2 295	2 716 844,79	1 213 973,44	1 073,43
Подрамники, рамы-Планшеты	3 850	3 466	2 550 475,59	1 170 236,14	662,46
Живопись-масло-профи-Мастер-Класс ЗХК	15 925	14 332	2 367 607,02	1 052 156,69	148,67
Сумки-на молнии ткань/кожа	2 949	2 640	2 184 469,42	983 199,08	740,75
Кисти-Масло-Синтетика	24 409	21 910	2 103 641,02	941 731,54	86,18
Графические материалы-маркеры-маркеры	15 459	13 904	1 948 429,28	872 331,25	126,04
Мольберты-Этюдники	593	536	1 855 947,64	849 012,66	3 129,76

Результаты приведены в таблице 13.

Таблица 13 – Выручка по филиалам

Филиал	Строки	Выручка
ТЗ-2 (Ленинградское шоссе)	431 372	58 750 803,24
ТЗ-1 (ул.Фадеева)	238 046	44 343 168,32
ТЗ-3 (Курский)	190 730	26 249 055,61
Багетная мастерская	848	1 399 096,07
РС (Распределительный склад)	249	788 679,75
Кафе-1 (ул.Фадеева)	3 977	207 762,35

Выполнены группировки по товарной группе и филиалу. Первая включает count, sum, avg и счётчик известной себестоимости. Два SQL-запроса

формируют рейтинг групп и динамику по месяцам; полный рейтинг SQL совпал с DataFrame API по двустороннему exceptAll.

```
SELECT product_group, COUNT(*) AS lines,
    COUNT(cost) AS cost_known,
    ROUND(SUM(amount), 2) AS revenue,
    ROUND(SUM(margin), 2) AS margin,
    ROUND(AVG(amount), 2) AS avg_line
FROM komus_sales
GROUP BY product_group
ORDER BY revenue DESC
```

```
SELECT month, COUNT(*) AS lines,
    ROUND(SUM(amount), 2) AS revenue,
    ROUND(SUM(discount), 2) AS discount
FROM komus_sales
GROUP BY month
ORDER BY month
```

Итоговая витрина содержит 437 товарных групп. Parquet и CSV сохранены в exports. Повторное чтение подтвердило количество строк и типы, изменение nullable зафиксировано отдельно. Расчётная маржа не является чистой прибылью: дополнительные расходы и налоги в источнике отсутствуют.

Результаты приведены в таблице 14.

Таблица 14 – Первые и последние месяцы продаж КОМУС

Месяц	Строки	Выручка	Скидка
2020-06	2 932	269 717,58	57 212,82
2020-07	13 218	1 317 945,84	208 144,55
2020-08	11 918	1 309 760,80	15 661,86
2021-10	107 459	17 447 678,83	314 085,35
2021-11	108 614	17 303 145,01	351 826,41
2021-12	126 773	23 024 328,52	509 723,89

Результаты приведены в таблице 15.

Таблица 15 – Схема итоговой витрины КОМУС

Поле	Тип до / после Parquet	Nullable до / после
product_group	string / string	False / True
lines	long / long	False / True
cost_known	long / long	False / True
revenue	double / double	True / True
margin	double / double	True / True
avg_line	double / double	True / True

```
group_stats.write.mode("overwrite").parquet(
    str(OUT / "pr3_komus_parquet"))
(group_stats.coalesce(1).write.mode("overwrite")
    .option("header", True).csv(str(OUT / "pr3_komus_csv")))
own_back = spark.read.parquet(str(OUT / "pr3_komus_parquet"))
```

Вывод: число строк после фильтра и обоих JOIN равно 865 222; исходный справочник прочитан без искажений. Суммарная выручка 131 738 565,34 совпадает с независимым пересчётом исходного архива. Получено 437 групп; лидер — Основы-холст на подрамнике-Базовый. SQL/API: 0 различий. После чтения Parquet совпали число строк, имена и типы полей; nullable проверен отдельно.

На рисунке 3 показан результат выполнения на стенде.

```
TIABD-Albakhtin-IV | student: Albakhtin I.V. | fresh run

TIABD PRACTICE 3 | KOMUS
Rows: 865222 Dimension unique: 21163
Null counts: {'receipt_id': 3934, 'purchase_date': 0, 'amount': 0,
 'cost': 86431, 'product_key': 0}
JOIN counts: {'before': 865222, 'left': 865222, 'inner': 865222, '
unmatched': 0}
Output groups: 437
CSV source mismatches: 0
Source dimension groups: 464
Roundtrip types: True
Top product groups:
Основы-холст на подрамнике-Базовый 6703 4407000.88
z-Уценка 37800 3296317.47
Живопись-акварель-наборы 6582 3182778.15
Бумага-Картон-пенокартон 2531 2716844.79
Подрамники, рамы-Планшеты 3850 2550475.59
_
```

Рисунок 3 – Реальные проверки данных КОМУС и обоих JOIN в Ubuntu

На рисунке 4 показан результат выполнения на стенде.

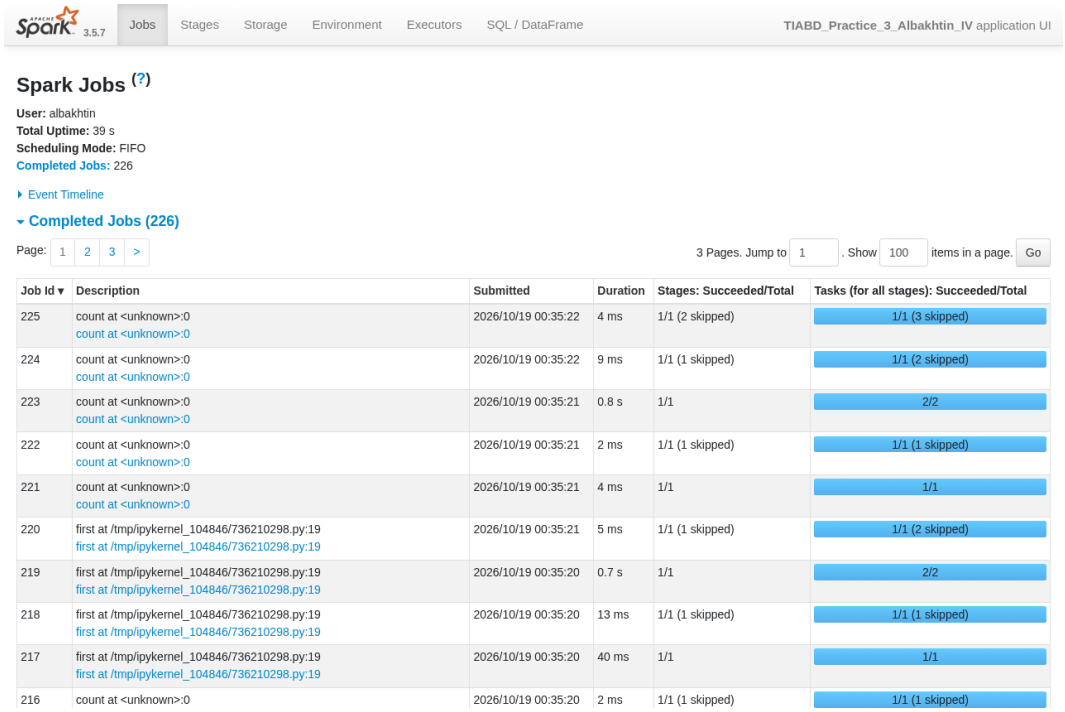


Рисунок 4 – Spark UI фактически выполненного приложения третьей практики

8 Воспроизводимость и выводы

Полный Run All выполнен 19.10.2026 00:35:27 МСК: новое ядро, 9 кодовых ячеек в порядке [1, 2, 3, 4, 5, 6, 7, 8, 9], ошибок 0, длительность 43,83 с. В последней ячейке выполнен spark.stop(). Сохраняются notebook, JSON результатов и выполнения, каталоги Parquet/CSV и свежий снимок Spark UI.

```
cd /home/albakhtin/tiabd/АлбахтинИВ/Spark
/home/albakhtin/tiabd/.venv/bin/python run_pr3.py
```

Проверка исходного CSV независимым парсером необходима наряду с SQL/API и round-trip: одинаковые агрегаты двух интерфейсов не обнаруживают ошибку, если оба читают уже искажённые поля.

1. Явная схема и преобразование timestamp сделали расчёты воспроизводимыми; автоматическая схема CSV не равна контракту данных.

2. Пропуски даты доставки связаны со статусом, поэтому массовое удаление незавершённых заказов исказило бы анализ.

3. LEFT JOIN Olist сохранил 112 650 позиций, включая 1 603 позиции неизвестной категории.

4. Полные результаты Spark SQL и DataFrame API совпали; обе реализации используют один вычислительный движок.

5. Все 865 222 продаж КОМУС сохранены, в том числе продажи без себестоимости; оценка маржи требует отдельного контроля охвата.

6. Справочник КОМУС покрывает все товарные ключи факта, поэтому INNER JOIN и LEFT JOIN дали одинаковое количество строк.

7. Parquet сохранил количество групп и типы полей, но nullable после чтения отличается от исходной схемы агрегатов.

Ограничения: локальная VM не проверяет работу физического кластера; восстановленная версия Olist отличается от SQLite примера по числу отзывов. Для КОМУС неизвестны полные условия формирования выборки, а неполная себестоимость ограничивает интерпретацию маржи. Выводы о сезонности и причинных связях по этим агрегатам не делаются.

Ответы на контрольные вопросы

1. Чем local[*] отличается от физического Spark-кластера и какие элементы программной модели при этом сохраняются?

local[*] использует доступные логические ядра одного узла. В физическом кластере executors размещены на разных узлах; DataFrame, partitions, задачи и ленивое планирование сохраняются в обоих режимах.

2. Каковы роли Driver и Executor в Spark-приложении?

Driver создаёт SparkContext, строит планы и планирует задачи. Executors выполняют задачи и хранят промежуточные данные; в local[*] вычислительная часть работает внутри локальной JVM.

3. Что такое partition и почему число partitions влияет на производительность?

Partition — логическая часть данных, над которой выполняется задача стадии. Малое число разделов ограничивает параллелизм, большое увеличивает расходы на планирование коротких задач.

4. В чём различие transformation и action? Приведите по три примера каждого типа.

Transformation строит план: select, filter, withColumn. Action требует результат: count, collect, запись через write. Одно действие может вызвать несколько внутренних Jobs.

5. Что означает lazy evaluation и почему время выполнения withColumn() без action не отражает стоимость вычислений?

Lazy evaluation откладывает вычисления до action. Создание withColumn формирует выражение, поэтому его время без action не измеряет чтение, вычисление и обмен данными.

6. Почему CSV не содержит строгой аналитической схемы и в каких случаях явный StructType предпочтительнее inferSchema?

CSV хранит текст и разделители, но не строгую схему. StructType полезен при стабильном контракте входа и позволяет избежать отдельного прохода inferSchema; некорректные значения всё равно требуют контроля.

7. Почему строку ISO-даты следует преобразовать в timestamp, даже если лексикографическое сравнение иногда даёт правильный порядок?

Timestamp задаёт временную семантику для datediff, month и сравнения дат. Строковое сравнение зависит от одинакового формата и не проверяет корректность даты.

8. Почему наличие null не означает, что строку нужно удалить? Как это показал пример order_delivered_customer_date?

Null может обозначать незавершённую операцию. В Olist большинство отсутствующих дат доставки относится к shipped, canceled и промежуточным статусам; только восемь пропусков относятся к delivered.

9. Чем select() отличается от filter(), а withColumn() — от простого выбора столбца?

Select выбирает столбцы и выражения, filter выбирает строки. WithColumn добавляет либо заменяет именованный столбец, а выбор существующего столбца не создаёт нового признака.

10. Почему после JOIN необходимо сравнивать число строк до и после операции?

Рост количества строк может указывать на неуникальный справочник или связь многие-ко-многим. Уменьшение при INNER JOIN может означать потерю фактов без подходящего ключа справа.

11. В чём содержательное различие INNER JOIN и LEFT JOIN при построении фактовой аналитической витрины?

INNER JOIN сохраняет только совпавшие ключи. LEFT JOIN сохраняет все левые факты и делает отсутствие справочной записи измеримым через null; выбор определяется целью витрины.

12. Почему COUNT(*) и COUNT(DISTINCT order_id) дают разные бизнес-смыслы для order_items?

COUNT(*) считает позиции заказа, COUNT(DISTINCT order_id) — уникальные заказы. Несколько позиций одного заказа дают разный результат этих двух агрегатов.

13. Как связаны Spark SQL и DataFrame API? Являются ли они разными вычислительными движками?

Оба интерфейса используют Spark SQL и Catalyst. SQL задаёт декларативный текст запроса, DataFrame API строит эквивалентные выражения; это не два независимых движка.

14. Почему Spark записывает DataFrame в каталог с part-файлами, а не обязательно в один файл?

Каждая выходная задача записывает свой part-файл. Каталог соответствует распределённой записи и включает служебные файлы, например _SUCCESS.

15. Почему coalesce(1) допустим для маленькой 74-строчной витрины, но является плохой общей стратегией записи больших данных?

Для 74 строк потеря параллелизма несущественна и один CSV удобен. Для большого факта coalesce(1) создаёт единственную задачу записи и узкое место.

16. Какие преимущества Parquet даёт последующим Spark-задачам по сравнению с CSV?

Parquet хранит типы, поддерживает колоночное чтение, сжатие и проталкивание фильтров. CSV требует восстановления типов; при чтении Parquet nullable может стать true даже при сохранении имён и типов полей.

Источники

1. Практическая работа №3. Распределенная модель обработки данных. Предоставленная методичка.

2. Транзакции КОМУС.zip. Предоставленный учебный архив; SHA-256 зафиксирован в source_manifest.json.

3. Apache Spark 3.5.7. Installation. https://spark.apache.org/docs/3.5.7/api/python/getting_started/install.html

4. Таблицы Olist из второй практики; версия и количество строк зафиксированы в source_manifest.json.